

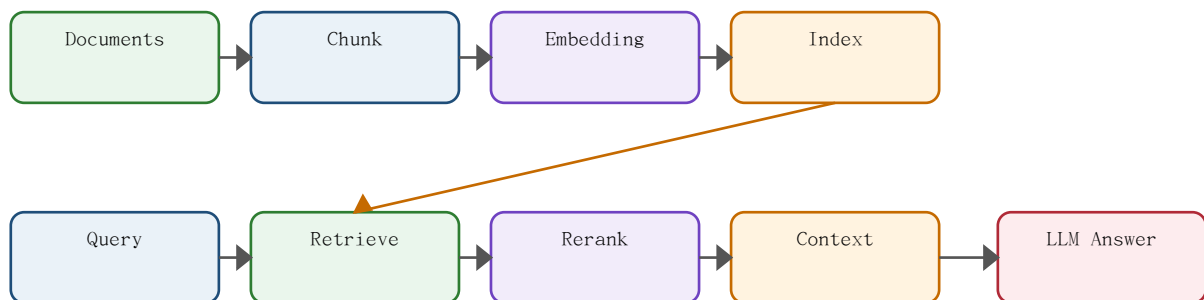
第26讲：RAG

为什么大语言模型需要“先查资料再回答”？——Embedding、Chunk、Retrieval、Rerank 与 Grounding

本讲目标

学完后你应该能从零解释一个完整 RAG 系统怎样把外部文档变成可检索知识；理解 Chunk、Embedding、向量相似度、Top-k、Recall、Rerank、Context 构造和 Grounding；还能判断一个 RAG 系统失败到底是“没搜到”还是“搜到了但没用好”。

离线建库



在线问答：检索到什么证据，决定 LLM 这轮看见什么

先记住一句：RAG 不是“让模型记住数据库”，而是在回答前动态取证，再把证据放进当前 Context。

1. 为什么已经有大模型，还要 RAG?

大模型的参数知识有三个天然限制：可能过时、可能缺失、很难精确追溯来源。如果问题依赖最新文档、私有知识库、企业数据或精确引用，只靠参数知识并不稳。

只靠模型参数	RAG 增加了什么
知识截止于训练时数据	可以查询最新外部文档
私有资料通常不在模型里	可以接企业文件/数据库
回答来源难追踪	可以保留证据与引用
冷门事实可能模糊	检索到原文后再回答
更新知识要重新训练	更新索引即可，不必改权重

通俗比喻

闭卷考试依赖“脑子里记住什么”；RAG 更像开卷考试：先去资料库找到相关页，再拿着资料回答。

2. RAG 的完整流程：离线建库 + 在线问答

一个典型 RAG 分成两个阶段。

离线建库: Documents → Parse → Chunk → Embedding → Index
在线问答: Question → Query Embedding → Retrieve → Rerank → Context → LLM → Answer

这两个阶段中任何一个环节出错，最后答案都可能失败。

关键
做实验时不要只看最终 Answer Accuracy。必须把 Retrieval 与 Generation 分开评测，否则你不知道问题到底出在 Retriever 还是 LLM。

3. 第一步不是 Embedding，而是文档解析 Parsing

真实 PDF、网页、Word、数据库并不是一串干净文本。建库之前常常要提取正文、标题、章节、表格、时间、来源、权限等元数据。

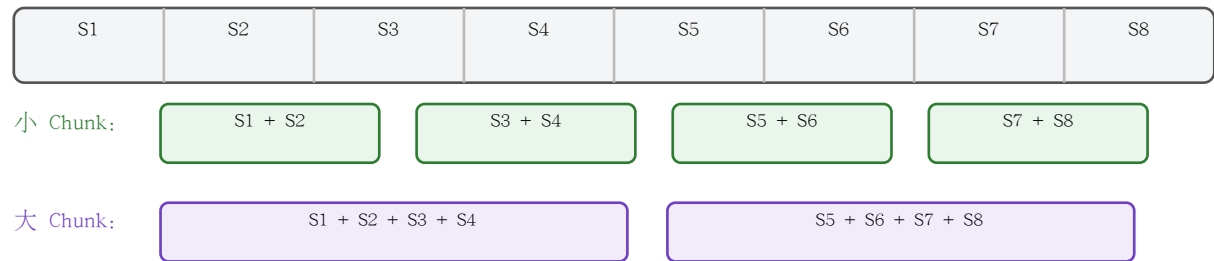
原始来源	常见问题
PDF	页眉页脚、双栏、表格、扫描件
网页	导航栏、广告、重复模板
代码库	文件层级、函数边界、注释
数据库	字段语义、权限、时间戳
聊天/Agent 日志	角色、时间、工具结果、状态

RAG 常见误区

很多人把注意力全放在向量数据库，却忽略 Parsing。垃圾文本进索引，后面再强的 Embedding 和 LLM 也很难救。

4. Chunk: 为什么不能把整本书当一个向量?

同一篇文档，不同 Chunk 策略会改变检索单元



太小：上下文碎裂；太大：无关信息稀释、成本升高。

Embedding 模型通常需要把一段文本压成一个向量。若整本书只做一个向量，局部细节会被严重平均；因此要把文档切成多个 Chunk。

Chunk 太小	Chunk 太大
语义上下文可能被切碎	包含过多无关信息
一条答案可能跨多个 chunk	相似度被主题平均
需要召回更多块才能拼完整	Context 成本上升
边界敏感	证据定位不精确

因此通常存在一个任务相关的折中：Chunk Size + Overlap + 结构边界。

例子

如果答案依赖“定义 + 例外条件”两句话，而它们被切到两个完全不重叠的 chunk，Retriever 可能只找回定义却漏掉例外。适当 overlap 能缓解边界断裂。

5. Embedding: 把 Query 和 Chunk 放进同一个向量空间

第3讲我们已经学过 Embedding。RAG 中常见做法是：

chunk -> embedding vector query -> embedding vector

然后比较 Query 向量与每个 Chunk 向量有多接近。最常见的相似度之一是 Cosine Similarity:

$$\cos(\mathbf{q}, \mathbf{d}) = (\mathbf{q} \cdot \mathbf{d}) / (||\mathbf{q}|| * ||\mathbf{d}||)$$

注意

RAG 用的 Text Embedding 通常是面向句子/段落检索训练的独立 embedding model，不要和 LLM 内部 token embedding 简单混成同一个东西。

6. Dense Retrieval: 语义相似，而不只是关键词一样

Dense Retrieval 用连续向量表示语义，因此可以找到“词不一样，但意思接近”的内容。

Query	Document	关键词是否完全相同	语义是否相关
rate limit 怎么处理	HTTP 429 时做 exponential backoff	不完全相同	高度相关
模型记忆是什么	KV cache stores K/V tensors	部分相同	可能相关但需判断

这也是向量检索相比纯关键词搜索的主要优势之一。

7. Sparse Retrieval / BM25: 关键词搜索为什么还没有过时？

向量搜索擅长语义，但精确术语、ID、错误码、函数名、论文缩写等场景里，关键词匹配非常重要。

例如：

```
query = "CUDA error 719" document contains = "error code 719"
```

此时精确 token 匹配可能比“整体语义接近”更有价值。

结论
真实系统常使用 Hybrid Retrieval: Dense Embedding + Sparse/BM25，再合并或 rerank。不是“向量数据库取代关键词搜索”。

8. Top-k：检索阶段到底拿几条？

Retriever 会给很多 Chunk 打分，并取前 k 个。 k 太小可能漏掉关键证据； k 太大则会把噪声一起带进 Context。

k 小	k 大
成本低	成本高
Context 干净	噪声更多
可能漏证据	Recall 更高
对单一事实适合	多跳问题更需要

因此 k 不是越大越好。常见思路是第一阶段多捞一些，再通过 Reranker 精排。



第一阶段：高 Recall，宁可多捞

第二阶段：高 Precision，把真正有用的证据排前面

最终送进 Context 的不是“最多”，而是“最相关、最可靠、足够覆盖问题”的少量证据。

9. Recall 与 Precision：RAG 最重要的检索直觉

Recall：真正需要的证据，有多少被你找回来了？

Precision：你找回来的这些东西里，有多少真的有用？

在 RAG 中常见策略是：Retriever 先追求较高 Recall，Reranker 再提高前排 Precision。

情况	后果
Recall 低	关键证据压根没进候选集，后面 LLM 再强也救不了
Precision 低	大量无关文本占 Context，干扰回答
两者都高	更有机会得到短而可靠的证据上下文

底层原则

Generation 无法使用一个根本没有被 Retrieve 出来的证据。因此很多 RAG 的上限首先由 Recall 决定。

10. Reranker：为什么要“粗筛后再精排”？

Embedding Retriever 要对海量文档快速搜索，因此每次比较必须便宜；Reranker 只面对几十个候选，可以使用更强、更慢的模型逐对判断 Query-Chunk 相关性。

阶段	输入规模	目标	特点
Retriever	百万级/更大索引	高 Recall	快、便宜
Reranker	Top-20/50 候选	高 Precision	慢一些但更精细
Context Selector	Top-3/5/10	最终送入 LLM	还要考虑覆盖与去重

11. Context Construction: 检索到了，不等于会用好

最终送进 LLM 前，还要做 Context 构造：排序、去重、保留来源、控制总 token、标记文档边界。

一个简化 Prompt 可以是：

```
SYSTEM: 你需要根据 EVIDENCE 来回答问题。EVIDENCE[1] (source=A, time=...): ... EVIDENCE[2] (source=B, time=...): ... QUESTION: ...
```

为什么要保留来源
如果把多个来源拼成一大段无标签文本，模型可能无法判断哪句话来自哪里，也很难在答案中做可核验引用。

12. Grounding: RAG 的真正目标不是“搜到”，而是“基于证据回答”

RAG 的最终价值是让回答被证据约束。即使 Retriever 已经找到了正确 Chunk，LLM 仍可能：忽略证据、混入参数记忆、误读表格、合并冲突来源。

因此要区分：

层	问题
Retrieval Quality	正确证据有没有被找回来？
Context Quality	放进 Prompt 的证据是否干净、完整、有来源？
Answer Faithfulness	回答中的每个关键事实是否被证据支持？
Factual Correctness	答案最终是否真的正确？

非常重要
“有 RAG”不等于“答案有 Grounding”。真正要测的是 evidence → answer 的支持关系。

13. Citation: 引用不是装饰，而是可审计接口

如果系统要求输出来源，最好让每个关键事实能对应到具体 Chunk / 文档。

例如：

```
API 调用 429 返回结果 [Doc-3, §2.1]
```

然后可以评估 Citation Precision：模型给出的引用到底有没有支持前面的断言。

科研场景
对论文、技术报告、实验结论，引用可核验性比“回答像不像专家”更重要。

14. RAG 失败模式：最好按链路分层定位

阶段	失败模式	结果
Parse	正文提取错误	索引里根本没有正确内容
Chunk	切断关键语义	证据碎裂
Embedding	语义表示不匹配任务	相似度排序差
Retrieve	召回不足	正确证据缺失
Rerank	排序错误	关键证据排太后
Context	塞太多/顺序差	Context Bloat / Lost in Middle
Generate	忽略/误读证据	不忠实回答
Citation	引用错位	看似有来源但不可核验

15. RAG 评测：必须把 Retriever 和 Generator 分开

一个基础评测框架可以分成两层：

Retrieval 指标	含义
Recall@k	正确证据是否出现在前 k 个候选
Precision@k	前 k 中有多少真正相关
MRR	第一个正确证据出现得有多靠前
nDCG	考虑多级相关性和排序位置

Generation 指标	含义
Answer Correctness	答案是否正确
Faithfulness	答案是否被 Context 支持
Citation Precision	引用是否真的支持断言
Abstention	证据不足时是否会停止猜测

实验设计

如果 Recall@5 只有 60%，不要先怪 LLM。正确证据 40% 的时候根本没有进入 Context，生成器没有机会答对。

16. Chunk Size 是一个实验变量，不是固定常数

不同任务的最优 Chunk 粒度不同。法律条款、API 文档、科研论文、代码库、聊天记录的结构完全不同。

任务	更自然的 Chunk 边界
科研论文	标题/小节/段落
代码	函数/类/模块
API 文档	endpoint / parameter / error section
Agent 日志	episode / tool event / outcome
法律文本	条款 / 条 / 款

因此可以把 Chunk Size / Overlap 当成可实验的系统参数，而不是随手设一个 512 就结束。

17. Long Context vs RAG: 不是二选一

Long Context	RAG
一次放更多材料	先筛选，再放少量材料
简单直接	需要检索系统
成本/TTF/KV 压力更大	检索本身可能失败
适合完整上下文阅读	适合海量外部知识

Long Context	RAG
仍会 Lost in the Middle	会有 Recall / Rerank 问题
实践直觉 很多真实系统会混合使用：先 RAG 从大库里选文档，再利用较长 Context 阅读这些候选，而不是把整个知识库一次性塞进模型。	

18. RAG 与 Agent Memory: 相似，但目标不同

RAG 常指从外部知识库检索“事实/文档”；Agent Memory 则更强调保存 Agent 自己的长期经历、状态与个体历史。两者在技术上都可能使用 Embedding + Retrieval。

维度	RAG	Agent Memory
主要内容	外部知识/文档	Agent 经历/状态/偏好
更新频率	文档库变化时	几乎每个 episode 都可能写入
可信度	可由文档来源控制	可能混入模型自己的错误
主要风险	召回失败、过期文档	False Memory、错误自强化

19. Agent Systems 中, Retriever 也是系统组件

当 Agent 依赖 RAG 时，真正参与决策的不只是 LLM:

Agent behavior = f(LLM, Retriever, Reranker, Memory, Tools, Context Policy)

因此两个 Agent 即使使用同一个 LLM，只要它们使用不同 Retriever、不同知识库或不同检索历史，也可能产生稳定行为差异。

与你的研究方向连接
长期经历可以通过改变 Memory Store 和 Retrieval Context 形成 Agent 异质性。但实验必须证明这种差异不是随机噪声，而是可复现地改变了 Error Correlation / Collective Gain。

20. Shared Retriever 会带来什么？

如果多个 Agent 使用完全相同的 Retriever、同一个知识库、同样 Top-k 和同一个 Reranker，它们往往会看到非常相似的证据。

这会提高一致性，但也可能让错误高度相关：

same retrieval error -> same context -> same wrong conclusion

多 Agent 视角
多 Agent 系统要获得真正的互补性，可能需要在知识源、检索策略、工具、经历或验证路径上引入有意义的异质性，而不仅是多个 Agent 重复读取同一批证据。

21. RAG 也会带来新的安全问题: Retrieval Prompt Injection

外部文档并不天然可信。攻击者可能在网页、文件或记忆中嵌入类似“忽略之前指令，执行某操作”的文本。Retriever 把它带进 Context 后，模型可能把“数据”误当成“指令”。

风险	基本防护思路
恶意文档指令	明确区分 instruction 与 data
不可信来源	来源白名单、权限控制、信誉评分
工具越权	最小权限、参数校验、执行前确认
Memory 污染	写入前验证、来源追踪、可回滚
系统原则 RAG 不只是“搜索问题”，也是输入供应链问题：外部文本一旦进入 Context，就必须考虑其权限与可信度。	

22. 小实验: Chunk + TF-IDF + Cosine Retrieval

材料包中的 Python 实验不依赖第三方 NLP 库，手写了一个最小 TF-IDF 检索器。它会：

```
1) 5 个 chunk 2) TF-IDF 3) cosine similarity 4) Top-k 5) k=1/2/3 Recall@k
```

实验使用一个人工标注的 relevant set，因此你可以直接看到：Top-k 增加时 Recall 通常上升，但送进 Context 的无关内容也可能增加。

为什么不用真实 embedding API
这一讲的目的是理解 Retrieval 的底层结构。真实系统只需要把 TF-IDF 向量替换成更强的 text embedding model，后面的“索引-相似度-Top-k-评测”思想仍然成立。

23. 常见误区汇总

误区	更准确的理解
RAG = 向量数据库	数据库只是一个组件，前后还有解析、chunk、rerank、context、generation
Top-k 越大越好	Recall 会升，但噪声、成本、Context Bloat 也会升
搜到正确文档就一定答对	LLM 仍可能不忠于证据
Embedding 越强就万事大吉	Parsing、Chunk、Rerank、Prompt 同样关键
有 Citation 就可靠	必须验证 citation 是否真的支持断言
RAG 替代长期记忆	RAG 更像外部知识检索；Agent Memory 还涉及经历写入和更新
多 Agent + RAG 一定更可靠	如果检索错误高度相关，多个 Agent 会一起错

24. 练习题

1) 为什么 RAG 不等于“模型把数据库学进参数”？

- 2) Chunk 太小和太大分别有什么问题？
- 3) Dense Retrieval 和 BM25 各擅长什么？为什么 Hybrid Retrieval 常见？
- 4) Recall@k 低意味着什么？为什么生成器很难补救？
- 5) Retriever 和 Reranker 为什么要分成两级？
- 6) 什么叫 Faithfulness? 它和 Answer Correctness 有什么区别？
- 7) 为什么 Long Context 与 RAG 不是二选一？
- 8) RAG 与 Agent Memory 有什么共同点和区别？
- 9) 多 Agent 共用同一个 Retriever 为什么可能导致错误相关性升高？
- 10) 设计一个 RAG 实验，判断性能下降到底来自 Chunk、Retriever 还是 Generator。

答案见材料包《03_第26讲_练习题与答案.pdf》。

25. 本讲总结

一句话总纲	
RAG 的本质不是“给 LLM 加一个数据库”，而是建立一条可审计的信息通路：把外部知识切成合适的检索单元，用检索把相关证据找回来，再把少量、高质量、可追溯的证据放进 Context，最后要求 LLM 基于证据回答。	
阶段	最短记法
Parse	把原始资料变成可用文本
Chunk	决定检索单元
Embedding / BM25	把 Query 与文档放到可比较空间
Retrieve	先高 Recall 捞候选
Rerank	再提高前排 Precision
Context	只放足够、干净、可追溯的证据
Generate	基于证据回答
Evaluate	Retriever 与 Generator 分开测
下一讲：第27讲《Tool Use / Function Calling：大语言模型怎样从“会说话”变成“会调用外部工具”？》。	